

Secure File Sharing System with AES Encryption

Future Interns – Cyber Security Internship (Task 3)

Name of Candidate: Aditya Choudhari

Internship Provider: Future Interns

Domain: Cyber Security

Task Title: Secure File Sharing System using AES Encryption

Date of Execution: 26th December 2025

Operating System: Windows

Programming Language: Python

Framework Used: Flask

1. Introduction

Secure file sharing is a critical requirement in modern organizations, especially in sectors such as healthcare, legal services, and corporate environments where sensitive data must be protected. Unauthorized access or data leakage can lead to serious security and compliance issues.

This task focuses on designing and implementing a **secure file sharing system** that encrypts files before storage and decrypts them only during authorized download. The project demonstrates practical use of cryptography and secure file handling techniques using Python and Flask.

2. Objective

The objectives of this task were:

- To develop a web-based file upload and download system
 - To implement AES encryption for securing files at rest
 - To ensure secure decryption of files during download
 - To demonstrate basic encryption key management
 - To understand secure file handling and data protection practices
-

3. Tools & Technologies Used

- **Python** – Core programming language
 - **Flask** – Backend web framework
 - **PyCryptodome** – Cryptographic library for AES encryption
 - **HTML** – Frontend interface
 - **Git & GitHub** – Version control (to be used after task completion)
-

4. System Architecture

The system follows a simple client-server architecture:

1. User uploads a file through a web interface
2. The backend encrypts the file using AES before storage
3. The encrypted file is stored on the server
4. On download request, the encrypted file is decrypted securely
5. The original file is delivered to the user

This ensures that files remain protected while stored on the server.

5. Encryption Methodology

AES (Advanced Encryption Standard)

- AES is a symmetric encryption algorithm widely used for data protection
- AES-256 encryption was implemented using PyCryptodome
- EAX mode was used to provide both confidentiality and integrity

Encryption Process

- File content is read as bytes
- Data is encrypted using AES before being saved
- Encrypted files are stored with a .enc extension

Decryption Process

- Encrypted file is read from storage
 - AES decryption is applied using the same secret key
 - Original file is reconstructed and sent to the user
-

6. Key Management Approach

- A 256-bit AES secret key was generated securely
- The key is stored server-side in a protected file (secret.key)
- The encryption key is never exposed to users
- All encryption and decryption operations are handled internally by the server

This approach ensures basic but effective key management suitable for an internship-level project.

7. File Upload and Download Workflow

Upload Workflow

- User selects a file through the web interface
- File is transmitted to the Flask backend
- File is encrypted using AES
- Encrypted file is saved in the uploads directory

Download Workflow

- User accesses a download endpoint
- Encrypted file is retrieved from storage
- File is decrypted securely
- Original file is delivered to the user

The system successfully handled filenames containing spaces during download.

8. Testing and Validation

The following tests were performed:

- File upload functionality tested successfully
- Encrypted .enc files verified to be unreadable in plain text
- Secure download tested using Flask endpoint
- Downloaded files matched original content exactly
- File integrity was maintained throughout the process

Screenshots and screen recordings were captured to demonstrate each step.

9. Security Considerations

- Files are encrypted before storage to protect data at rest
 - Encryption keys are stored securely on the server
 - Only encrypted files are stored on disk
 - Decryption occurs only during download requests
-

10. Limitations and Future Improvements

- Decrypted files are temporarily written during download
- No user authentication mechanism is implemented
- Key rotation is not implemented

Future Improvements:

- Stream decrypted files without writing to disk
 - Implement user authentication and access control
 - Store encryption keys using environment variables or key vaults
-

11. Challenges Faced

- Setting up Flask application structure correctly
- Managing encryption and decryption logic
- Handling file paths and filenames with spaces
- Debugging template and routing issues

These challenges improved understanding of secure application development.

12. Learning Outcomes

Through this task, I gained:

- Hands-on experience with AES encryption
 - Understanding of secure file handling techniques
 - Practical knowledge of Flask web development
 - Awareness of key management and security best practices
 - Experience in building a real-world security-focused application
-

13. Conclusion

This task successfully demonstrated the implementation of a secure file sharing system using AES encryption. Files were protected at rest and securely decrypted during download, ensuring confidentiality and integrity. The project provided valuable exposure to cryptography, secure application development, and real-world cybersecurity practices.

14. Screenshots & Evidence

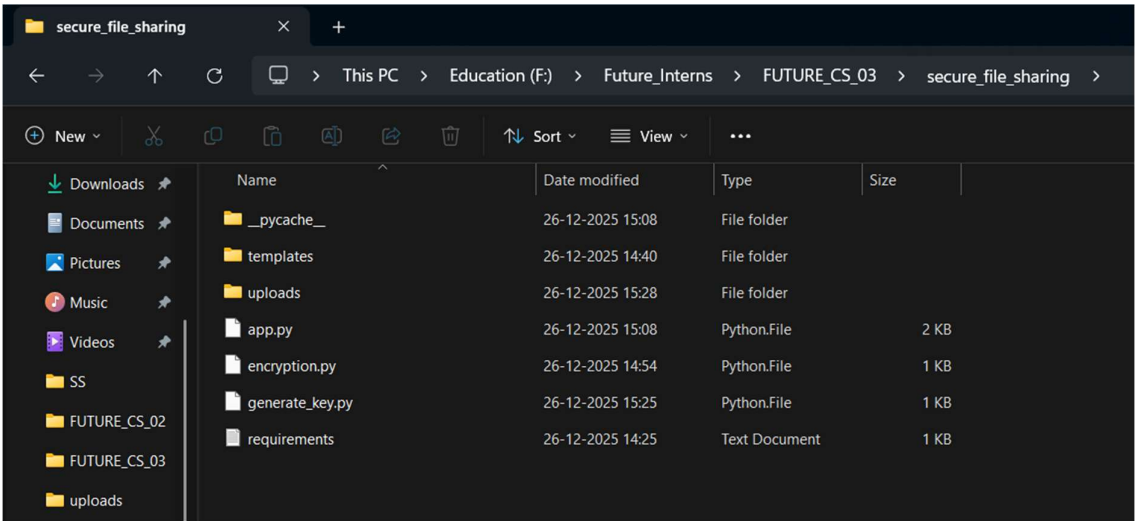
Screenshots and screen recordings were captured throughout the implementation and testing of the secure file sharing system to validate functionality and security controls.

The following evidence was collected:

- Screenshot of the project directory structure showing Flask application files and encryption components
- Screenshot of the Flask application running on localhost (<http://127.0.0.1:5000>)
- Screenshot of the file upload interface used to upload files for encryption
- Screenshot showing the encrypted file (.enc) stored in the upload's directory, confirming encryption at rest
- Screenshot of the encrypted file opened in a text editor, demonstrating unreadable encrypted content
- Screenshot of the secure download process using the Flask download endpoint
- Screenshot of the successfully decrypted file showing original content after download

These screenshots and recordings demonstrate that files were securely encrypted before storage and correctly decrypted only during authorized download, ensuring data confidentiality and integrity.

1. All Folders



2. App.py (Backend programming)

```
app.py x
app.py > ...
1 from flask import Flask, render_template, request, send_file
2 import os
3 from encryption import encrypt_file, decrypt_file
4
5 app = Flask(__name__)
6
7 UPLOAD_FOLDER = "uploads"
8 os.makedirs(UPLOAD_FOLDER, exist_ok=True)
9
10 @app.route("/", methods=["GET", "POST"])
11 def upload_file():
12     if request.method == "POST":
13         uploaded_file = request.files["file"]
14
15         if uploaded_file.filename == "":
16             return "No file selected"
17
18         file_data = uploaded_file.read()
19         encrypted_data = encrypt_file(file_data)
20
21         encrypted_path = os.path.join(UPLOAD_FOLDER, uploaded_file.filename + ".enc")
22         with open(encrypted_path, "wb") as f:
23             f.write(encrypted_data)
24
25         return "File uploaded and encrypted successfully"
26
27     return render_template("upload.html")
28
29
30 @app.route("/download/<filename>")
31 def download_file(filename):
32     encrypted_path = os.path.join(UPLOAD_FOLDER, filename)
33
34     with open(encrypted_path, "rb") as f:
35         encrypted_data = f.read()
36
37     decrypted_data = decrypt_file(encrypted_data)
38
39     decrypted_path = encrypted_path.replace(".enc", "")
40     with open(decrypted_path, "wb") as f:
41         f.write(decrypted_data)
42
43     return send_file(decrypted_path, as_attachment=True)
44
45
46 if __name__ == "__main__":
47     app.run(debug=True)
48
```

3. Encryption Logic

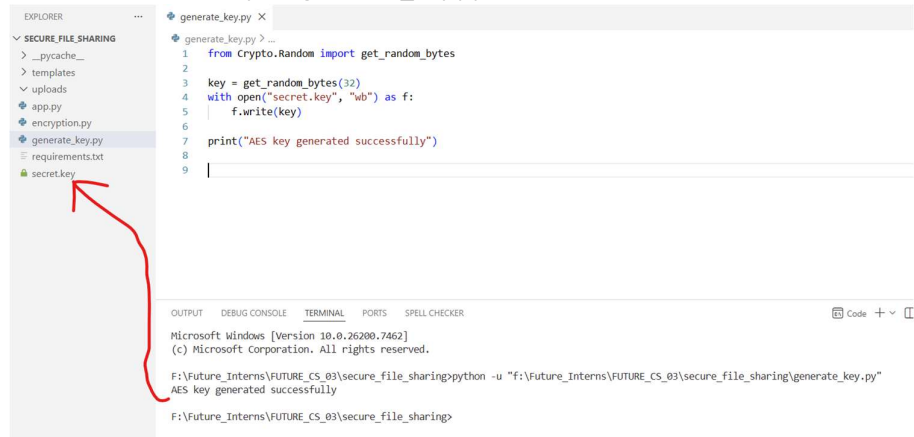
```
encryption.py > ...
1 from Crypto.Cipher import AES
2 from Crypto.Random import get_random_bytes
3
4 KEY_FILE = "secret.key"
5
6 def load_key():
7
8     with open(KEY_FILE, "rb") as f:
9         return f.read()
10
11 def encrypt_file(file_data):
12
13     key = load_key()
14     cipher = AES.new(key, AES.MODE_EAX)
15     ciphertext, tag = cipher.encrypt_and_digest(file_data)
16
17     encrypted_data = cipher.nonce + tag + ciphertext
18     return encrypted_data
19
20 def decrypt_file(encrypted_data):
21
22     key = load_key()
23
24     nonce = encrypted_data[:16]
25     tag = encrypted_data[16:32]
26     ciphertext = encrypted_data[32:]
27
28     cipher = AES.new(key, AES.MODE_EAX, nonce=nonce)
29     data = cipher.decrypt_and_verify(ciphertext, tag)
30     return data

```

4. AES Key generation logic

```
generate_key.py X
generate_key.py > ...
1  from Crypto.Random import get_random_bytes
2
3  key = get_random_bytes(32)
4  with open("secret.key", "wb") as f:
5      f.write(key)
6
7  print("AES key generated successfully")
8
```

5. Generation of AES key via generate_key.py



```
EXPLORER
SECURE_FILE_SHARING
  > __pycache__
  > templates
  > uploads
  app.py
  encryption.py
  generate_key.py
  requirements.txt
  secret.key

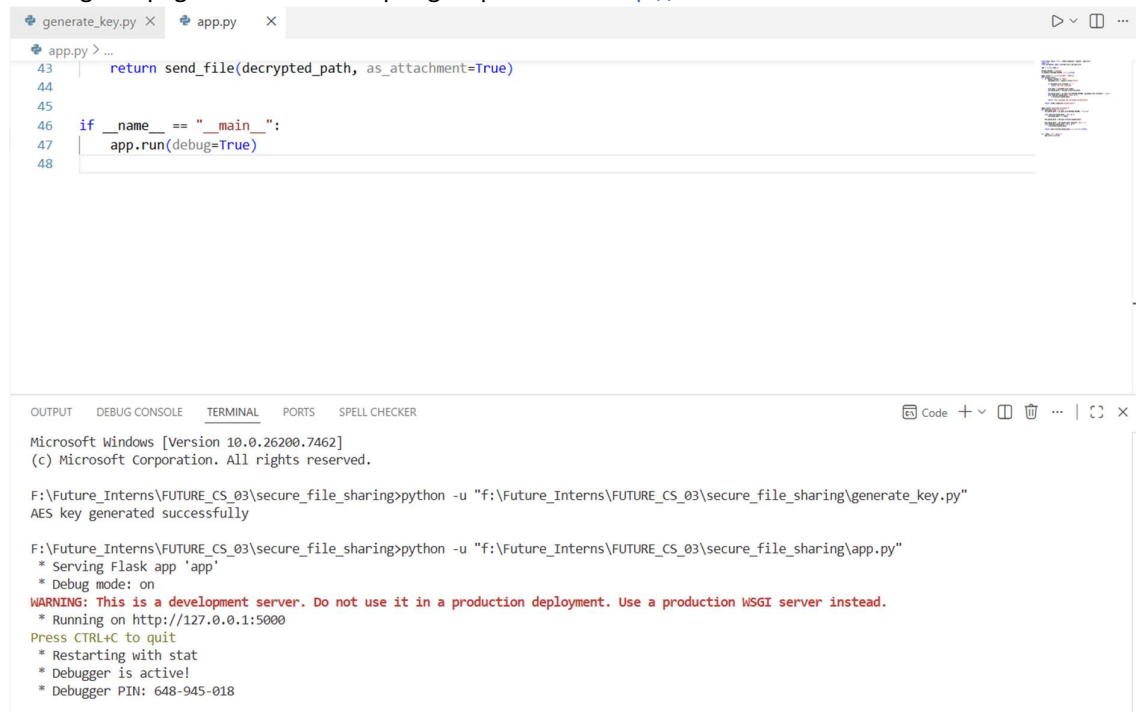
generate_key.py X
generate_key.py > ...
1  from Crypto.Random import get_random_bytes
2
3  key = get_random_bytes(32)
4  with open("secret.key", "wb") as f:
5      f.write(key)
6
7  print("AES key generated successfully")
8
9

OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER
Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

F:\Future_Interns\FUTURE_CS_03\secure_file_sharing>python -u "f:\Future_Interns\FUTURE_CS_03\secure_file_sharing\generate_key.py"
AES key generated successfully

F:\Future_Interns\FUTURE_CS_03\secure_file_sharing>
```

6. Making webpage accessible accepting responses via <http://127.0.0.1:5000>



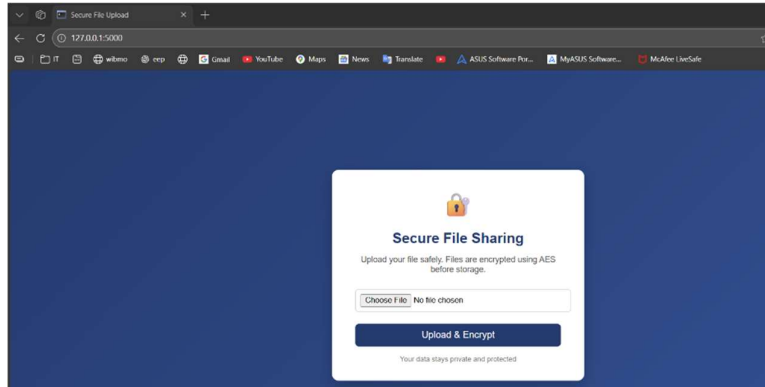
```
generate_key.py X  app.py X
app.py > ...
43  return send_file(decrypted_path, as_attachment=True)
44
45
46  if __name__ == "__main__":
47      app.run(debug=True)
48
```

```
OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER
Microsoft Windows [Version 10.0.26200.7462]
(c) Microsoft Corporation. All rights reserved.

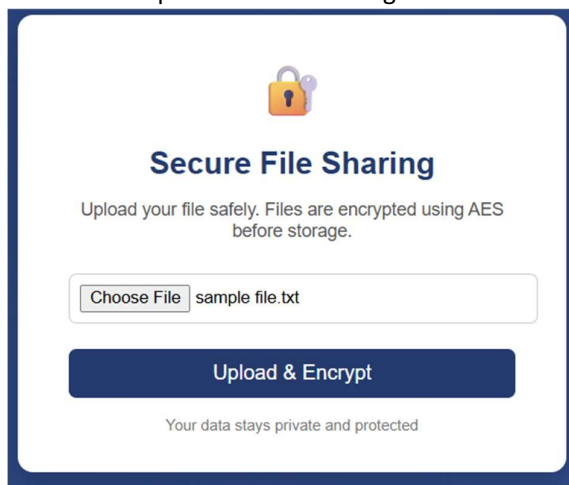
F:\Future_Interns\FUTURE_CS_03\secure_file_sharing>python -u "f:\Future_Interns\FUTURE_CS_03\secure_file_sharing\generate_key.py"
AES key generated successfully

F:\Future_Interns\FUTURE_CS_03\secure_file_sharing>python -u "f:\Future_Interns\FUTURE_CS_03\secure_file_sharing\app.py"
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 648-945-018
```

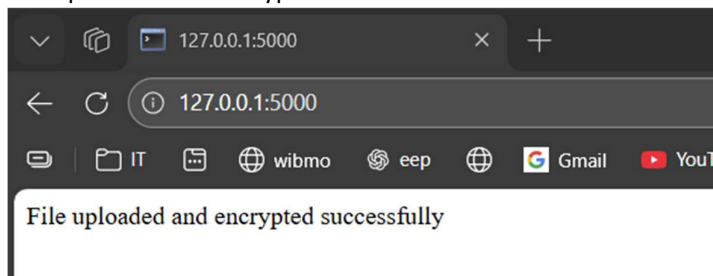
7. User Interface for accepting files for further sharing



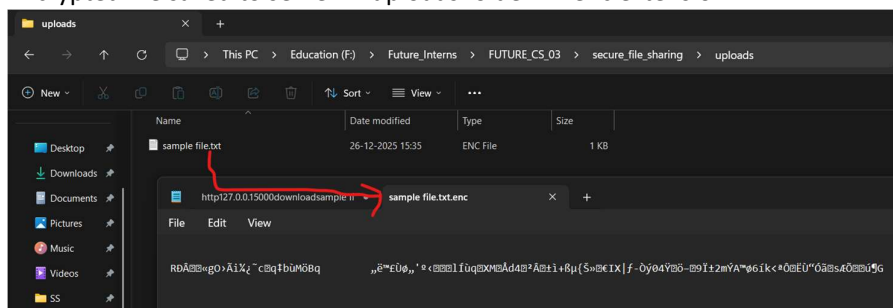
8. Chosen sample file.txt for sharing



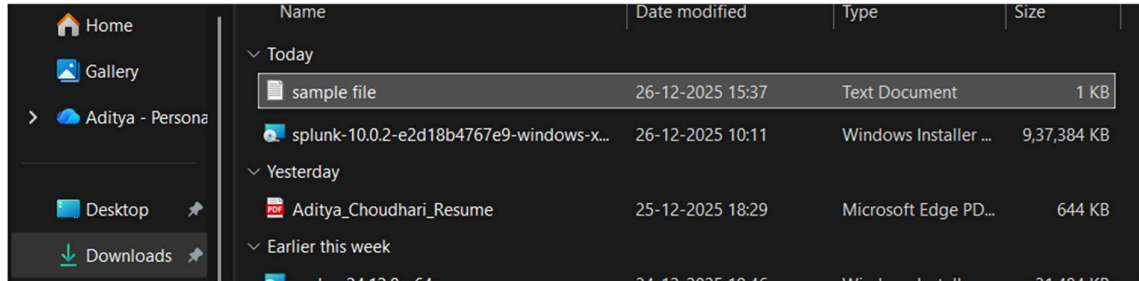
9. File uploaded and encrypted



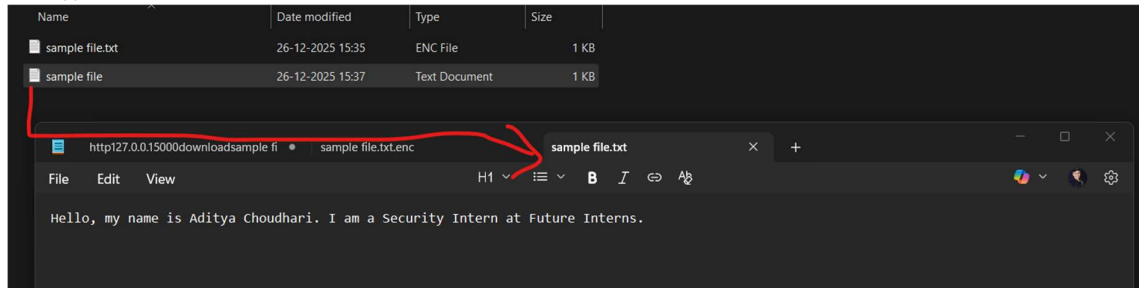
10. Encrypted file saved to server in uploads folder in .enc extension



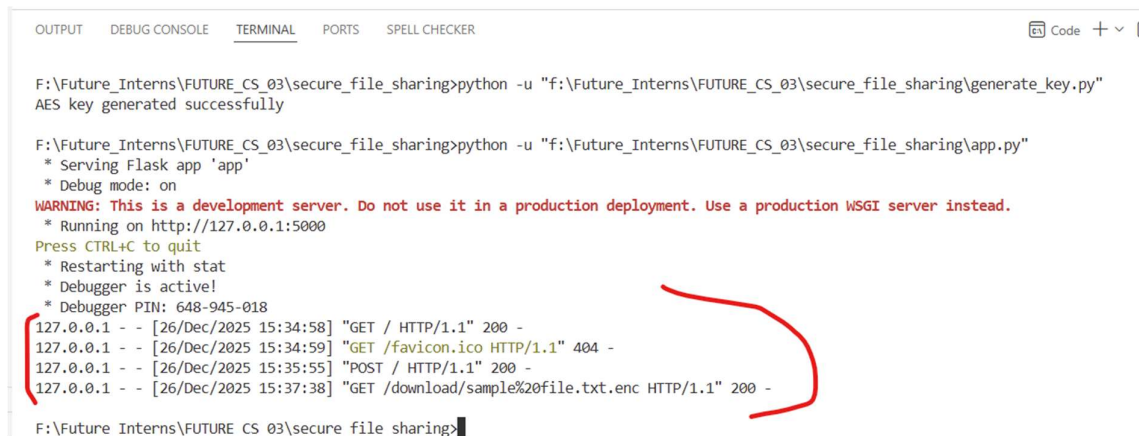
11. Visit <http://127.0.0.1:5000/download/sample.file.txt.enc> to download decrypted file



12. Decrypted file



13. All communication records



POST & GET request while file sharing.